

基于 ESP32-C3 与 MLX90640 的 热成像相机系统开发流程说明

嵌入式采集 · WiFi Web 服务 · 热成像可视化 · 数据记录

项目定位

本项目围绕 MLX90640 热红外阵列传感器与 ESP32-C3 开发板，完成从 I2C 采集、无线接入、HTTP 数据接口到浏览器热图显示的完整软件链路。当前报告用于记录开发流程、仓库文件、硬件连接方式和调试验证方法。

主控平台：ESP32-C3，集成 WiFi/BLE，可承担主控与 Web Server

热成像器件：MLX90640，32 x 24 红外阵列，I2C 地址 0x33

开发环境：VS Code + PlatformIO + ESP-IDF

仓库地址：<https://github.com/1909899270h-spec/esp32c3-mlx90640-thermal-camera>

1. 系统总体链路

系统的核心不是单独读取一个传感器，而是把“采集、网络、网页、记录”连成一条可演示的数据链路。
ESP32-C3 在其中同时承担主控、无线通信和轻量级服务器角色。

系统总体数据链路

从红外阵列采样到浏览器热图显示，ESP32-C3 同时承担主控、无线接入和小型 Web Server。



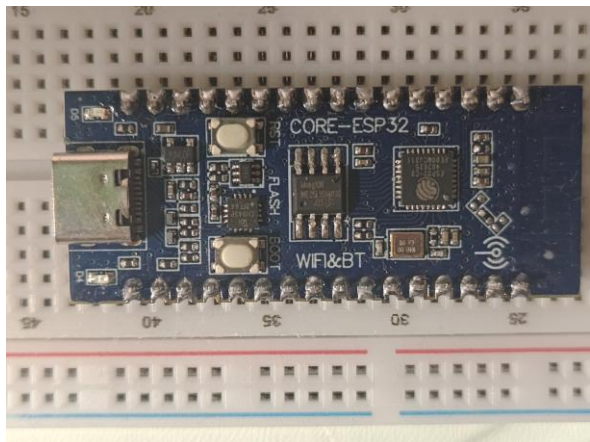
核心接口：GPIO4/SDA、GPIO5/SCL、3.3V、GND；网页访问：AP 模式默认 192.168.4.1。

图 1 从 MLX90640 到浏览器热图的完整数据链路

- 传感器侧：MLX90640 输出 32 x 24 阵列温度数据，通过 I2C 与主控通信。
- 主控侧：ESP32-C3 完成采样调度、温度计算、数据缓存和 HTTP 接口返回。
- 显示侧：浏览器网页通过 JSON 接口刷新 Canvas 热图，并支持色温条与 CSV 数据记录。

2. 硬件组成与连接

硬件搭建采用面包板与杜邦线完成，便于在调试阶段反复调整接线。系统最小硬件包括 ESP32-C3 开发板、MLX90640 红外阵列传感器、I2C 上拉电阻和供电去耦电容。



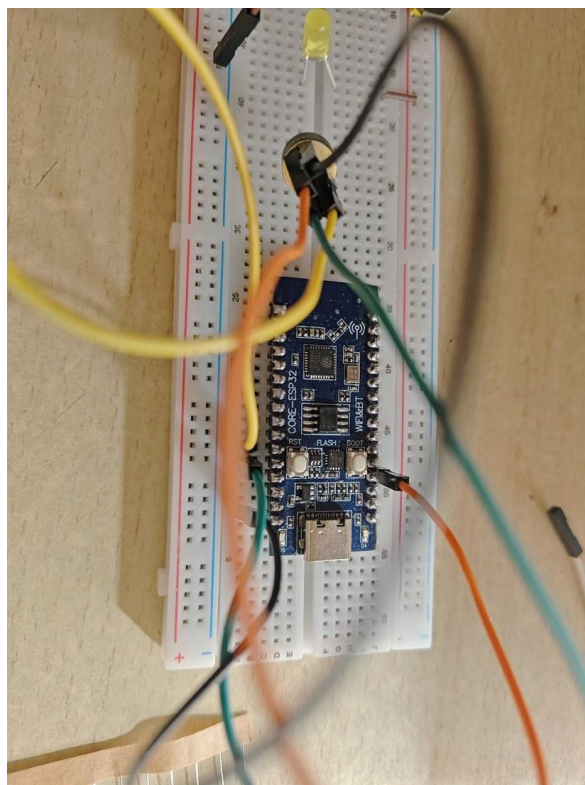
ESP32-C3 开发板正面



MLX90640 裸传感器



开发板背面引脚标识



面包板接线过程

核心接线

传感器端	ESP32-C3 端	作用	备注
MLX90640 SDA	ESP32-C3 GPIO4	I2C 数据线	5kΩ 上拉到 3.3V
MLX90640 SCL	ESP32-C3 GPIO5	I2C 时钟线	5kΩ 上拉到 3.3V
MLX90640 VDD	ESP32-C3 3.3V	传感器供电	建议 10uF 去耦电容
MLX90640 GND	ESP32-C3 GND	公共地	必须与主控共地

电阻与电容的作用

SDA/SCL 是开漏/开集电极风格的总线信号，需要通过上拉电阻保持高电平；10uF 电容用于降低供电波动，减少传感器读数异常和 I2C 通信不稳定。

```
src > C main.c > ...
110 static const char* html_script = "<script src=\"/web_script.js\"></script></body></html>";
111
112 // I2C 配置
113 #define I2C_MASTER_SCL_IO GPIO_NUM_5 /*!< GPIO number used for I2C master clock */
114 #define I2C_MASTER_SDA_IO GPIO_NUM_4 /*!< GPIO number used for I2C master data */
115 #define I2C_MASTER_NUM I2C_NUM_0 /*!< I2C port number for master dev */
116 #define I2C_MASTER_FREQ_HZ 400000 /*!< I2C master clock frequency (400kHz) */
117 #define I2C_MASTER_TX_BUF_DISABLE 0 /*!< I2C master doesn't need buffer */
118 #define I2C_MASTER_RX_BUF_DISABLE 0 /*!< I2C master doesn't need buffer */
119
120 // MLX90640 配置
121 const uint8_t MLX90640_I2C_ADDR = 0x33; // MLX90640 默认I2C地址
122 #define TA_SHIFT 8 // 来自库示例的环境温度偏移
123 static paramsMLX90640 mlx90640_params;
124 static float mlx90640_temperatures[THERMAL_DATA_SIZE];
125 static uint16_t eeMLX90640[832]; // EEPROM数据缓冲区
126 static bool mlx90640_initialized = false;
127
128 // 添加新的数据缓冲区和任务控制变量
129 static uint16_t mlx90640Frame[834]; // 静态以避免栈溢出
130 static SemaphoreHandle_t frameMutex = NULL;
131 static TaskHandle_t frameCaptureTaskHandle = NULL;
132 static bool taskRunning = false;
133 static float emissivity = 0.95;
134 static int mlx90640_init_status = -999;
135 static int mlx90640_eeprom_status = -999;
136 static int mlx90640_extract_status = -999;
137 static int mlx90640_refresh_status = -999;
138 static int mlx90640_chess_status = -999;
139 static int last_frame_status = 0;
140 static uint32_t frame_ok_count = 0;
141 static uint32_t frame_fail_count = 0;
142 static esp_err_t last_i2c_error = ESP_OK;
143 static uint32_t i2c_error_count = 0;
144 static uint8_t last_i2c_error_addr = 0;
145 static uint16_t last_i2c_error_reg = 0;
```

图 2 I2C 与 MLX90640 引脚配置说明

3. 开发环境与仓库结构

开发链路以 VS Code + PlatformIO 为主。PlatformIO 负责工程依赖、ESP-IDF 框架、编译、串口烧录、SPIFFS 文件系统上传和串口监视，适合把嵌入式代码与网页资源放在同一个工程中管理。

```
1 ; PlatformIO Project Configuration File
2 ;
3 ; Build options: build flags, source filter
4 ; Upload options: custom upload port, speed and extra flags
5 ; Library options: dependencies, extra library storages
6 ; Advanced options: extra scripting
7 ;
8 ; Please visit documentation for the other options and examples
9 ; https://docs.platformio.org/page/projectconf.html
10
11 [env:esp32-c3-devkitm-1]
12 platform = espressif32@6.9.0
13 board = esp32-c3-devkitm-1
14 framework = espidf
15 monitor_speed = 115200
16 upload_speed = 921600
17 monitor_filters = direct
18 board_build.flash_size = 4MB
19 board_build.partitions = partitions.csv
20 board_build.filesystem = spiffs
21
22 ; 添加自定义上传目标
23 extra_scripts = pre:extra_script.py
24
```

图 3 VS Code + PlatformIO 的工程配置与烧录入口

仓库关键文件

esp32c3-mlx90640-thermal-camera/	
├─ platformio.ini	# PlatformIO 工程、板卡、烧录与文件系统配置
├─ src/main.c	# ESP32-C3 主程序: I2C、WiFi AP、HTTP Server、接口逻辑
├─ data/web_script.js	# 前端脚本: 热图绘制、色温条、按钮交互、CSV 导出
├─ lib/MLX90640/	# MLX90640 API 与参数计算代码
├─ tools/mock_thermal_server.py	# 无硬件调试用的本地网页与数据生成服务
├─ partitions.csv	# Flash 分区, 支持 SPIFFS 网页资源
├─ README.md	# 仓库说明、接线、烧录、演示入口
└─ deliverables/	# 答辩 PPT、开发流程 DOCX/PDF 与生成脚本

- 固件端网页的 HTML/CSS 由 src/main.c 中的字符串返回，前端交互逻辑主要放在 data/web_script.js。
- 调试端 tools/mock_thermal_server.py 可在没有传感器稳定数据时验证热图、按钮、CSV 下载等网页功能。
- README 与 deliverables 记录了硬件连接、烧录流程、文档和演示材料，便于后续开源维护。

4. 软件架构与关键代码

软件部分按“采集任务 - 数据缓存 - HTTP 接口 - 网页渲染”的方式组织。这样可以把传感器采样与网页请求解耦，浏览器访问数据时读取最近一帧，而不是每次请求都阻塞等待传感器。

关键代码截图

```
src > C main.c > ...
263
264 // MLX90640 传感器初始化
265 static esp_err_t mlx90640_sensor_init(void) {
266     int status;
267     mlx90640_initialized = false;
268     mlx90640_init_status = -1;
269
270     status = MLX90640_DumpEE(MLX90640_I2C_ADDR, eeMLX90640);
271     mlx90640_eeprom_status = status;
272     if (status != 0) {
273         ESP_LOGE(TAG, "Failed to load EEPROM (status = %d)", status);
274         mlx90640_init_status = status;
275         return ESP_FAIL;
276     }
277     ESP_LOGI(TAG, "EEPROM loaded successfully.");
278
279     status = MLX90640_ExtractParameters(eeMLX90640, &mlx90640_params);
280     mlx90640_extract_status = status;
281     if (status != 0) {
282         ESP_LOGE(TAG, "Failed to extract parameters (status = %d)", status);
283         mlx90640_init_status = status;
284         return ESP_FAIL;
285     }
286     ESP_LOGI(TAG, "Parameters extracted successfully.");
287
288     // 提高刷新率到8Hz (0x04) 或 16Hz (0x05)
289     status = MLX90640_SetRefreshRate(MLX90640_I2C_ADDR, 0x05); // 16Hz
290     mlx90640_refresh_status = status;
291     if (status != 0) {
292         ESP_LOGE(TAG, "Failed to set refresh rate (status = %d)", status);
293         // 继续，即使刷新率设置失败
294     } else {
295         ESP_LOGI(TAG, "Refresh rate set to 16Hz.");
296     }
297
298     // 设置为棋盘模式 (Chess mode)
299     status = MLX90640_SetChessMode(MLX90640_I2C_ADDR);
```

图 4 MLX90640 初始化：I2C 参数、传感器参数读取与初始化状态判断


```

319 // 新增：持续捕获帧的任务
320 static void frame_capture_task(void *pvParameters) {
321     float tr; // 反射温度
322
323     while (taskRunning) {
324         // 获取帧数据
325         int status = MLX90640_GetFrameData(MLX90640_I2C_ADDR, mlx90640Frame);
326         last_frame_status = status;
327         if (status < 0) {
328             frame_fail_count++;
329             ESP_LOGE(TAG, "GetFrameData failed with status: %d", status);
330             vTaskDelay(pdMS_TO_TICKS(10));
331             continue;
332         }
333         frame_ok_count++;
334
335         float ta = MLX90640_GetTa(mlx90640Frame, &mlx90640_params);
336         tr = ta - TA_SHIFT; // 根据环境温度计算反射温度
337
338         // 获取互斥锁并更新温度数据
339         if (xSemaphoreTake(frameMutex, pdMS_TO_TICKS(100)) == pdTRUE) {
340             MLX90640_CalculateTo(mlx90640Frame, &mlx90640_params, emissivity, tr, mlx90640_temperatures);
341             xSemaphoreGive(frameMutex);
342         }
343
344         // 不需要两帧间等待太久，传感器本身会按照配置的刷新率工作
345         vTaskDelay(pdMS_TO_TICKS(10)); // 短暂延迟以避免任务占用过多CPU
346     }
347
348     vTaskDelete(NULL);
349 }
350

```

图 5 温度采集任务：持续读取帧数据、计算 To 数组并更新共享缓存

```

535 // HTTP GET处理函数 - 热成像数据
536 static esp_err_t thermal_data_get_handler(httpd_req_t *req)
537 {
538     if (!mlx90640_initialized) {
539         ESP_LOGE(TAG, "MLX90640 not initialized yet.");
540         httpd_resp_send_500(req);
541         return ESP_FAIL;
542     }
543
544     // 确保帧捕获任务正在运行
545     if (!taskRunning) {
546         start_frame_capture_task();
547     }
548
549     // 分配JSON缓冲区 - 使用静态缓冲区以避免频繁内存分配
550     static char json_buffer[JSON_BUFFER_SIZE];
551
552     // 从最新的温度数据生成JSON
553     if (xSemaphoreTake(frameMutex, pdMS_TO_TICKS(100)) == pdTRUE) {
554         int current_pos = 0;
555
556         // 使用更快的方式构建JSON
557         json_buffer[current_pos++] = '[';
558
559         for(int i = 0; i < THERMAL_DATA_SIZE; i++) {
560             // 使用1位小数来减少数据大小，同时保持足够的精度
561             int written = snprintf(json_buffer + current_pos, JSON_BUFFER_SIZE - current_pos,
562                                   "%s%.1f", i == 0 ? "" : ",", mlx90640_temperatures[i]);
563
564             if (written < 0 || written >= JSON_BUFFER_SIZE - current_pos) {
565                 ESP_LOGE(TAG, "JSON buffer overflow");
566                 xSemaphoreGive(frameMutex);
567                 httpd_resp_send_500(req);
568                 return ESP_FAIL;
569             }
570             current_pos += written;
571         }
572         json_buffer[current_pos++] = ']';
573     }
574
575     httpd_resp_send(req, json_buffer, HTTPD_RESP_USE_STRLEN);
576     return ESP_OK;
577 }

```

图 6 网页数据接口：将最新温度数组编码为 JSON 并返回给浏览器

5. 网页热图与局域网访问

网页端由 HTML、CSS 与 JavaScript 构成：HTML 定义按钮和画布，CSS 控制深色界面与控制栏样式，JavaScript 负责定时请求数据、Canvas 热图绘制、色温条显示、开始检测、导入数据、下载 CSV 和全屏显示。

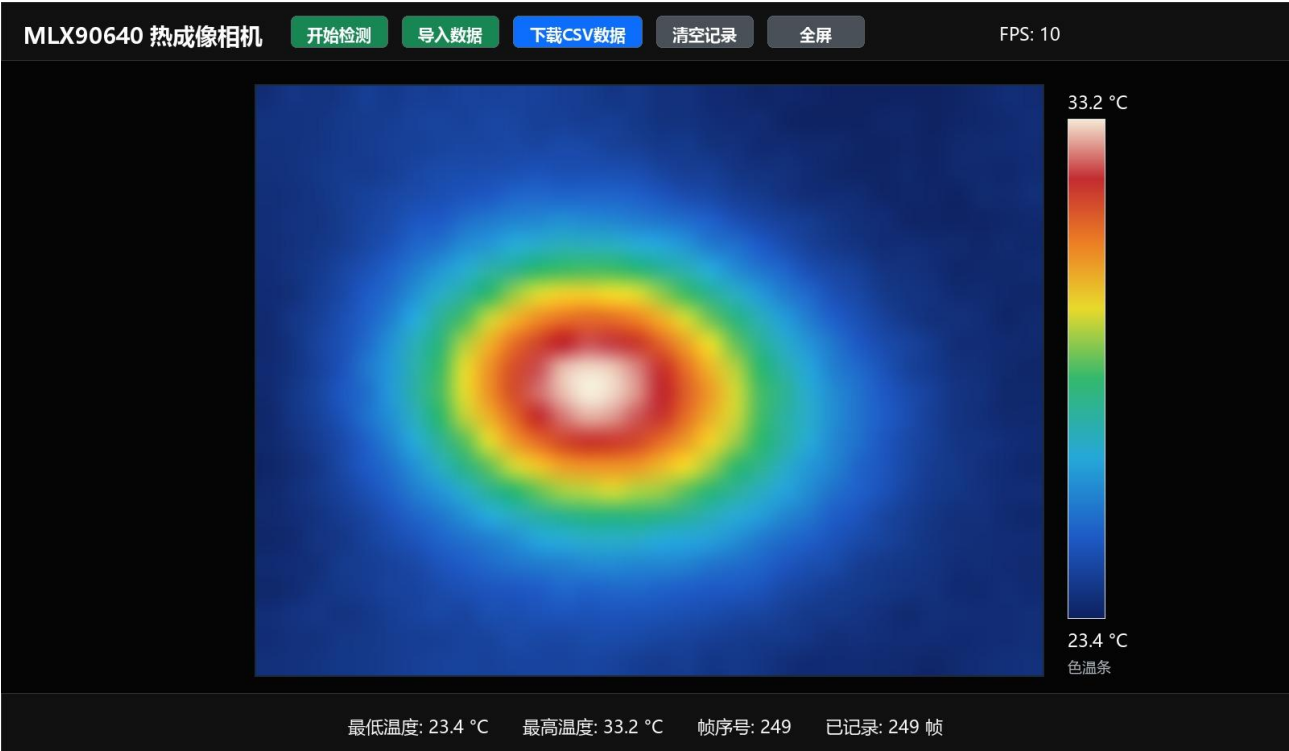


图 7 网页热图调试效果：用于验证 Canvas、色温条、按钮交互和数据记录逻辑

为什么只能在局域网访问

- AP 模式下，ESP32-C3 自己建立 WiFi 热点，手机或电脑连接该热点后访问 192.168.4.1。
- STA 模式下，ESP32-C3 连接路由器，浏览器访问的是路由器分配给 ESP32 的局域网 IP。
- 这些地址属于局域网私有地址，不是公网地址；手机切换到其他网络后无法直接访问，这是网络结构原因，不是网页代码错误。

6. 调试与问题定位思路

调试时不能只看网页是否出现热图，需要分层判断问题发生在供电、总线、数据计算还是网页显示。下面的三层判断可以覆盖大部分异常情况。

第一层：硬件层

- 确认 ESP32-C3 Type-C 供电正常，电脑设备管理器能识别 COM 口。
- 确认 MLX90640 的 VDD 接 3.3V、GND 共地，杜邦线没有松动或短接。
- 确认 SDA/SCL 上拉电阻和 10uF 去耦电容接入合理。

第二层：总线层

- 通过 I2C scan 或串口日志判断是否能扫描到 MLX90640 地址 0x33。
- 如果扫描不到，优先检查 GPIO4/GPIO5 是否接反、上拉是否有效、传感器引脚方向是否一致。

第三层：数据层与网页层

- 观察网页 min/max/avg 是否随热源变化，若始终为空或固定值，说明采集链路仍未稳定。
- 若数据正常但图像不动，检查 /thermal-data JSON、Canvas 绘制和前端刷新逻辑。
- CSV 下载只记录已接收到的数据帧，不应伪造未经验证的真实采样结果。

7. 当前完成情况与后续优化

当前状态

目前已完成 ESP32-C3 Web 服务、网页热图显示、数据接口、CSV 导出逻辑、调试数据链路和硬件接线方案；MLX90640 实时采集链路正在进行稳定性验证。

已完成内容

- PlatformIO 工程、ESP32-C3 固件编译、串口烧录与 SPIFFS 文件系统上传流程已经建立。
- 网页端支持热图可视化、色温条、开始检测、导入调试数据、全屏显示与 CSV 下载。
- GitHub 仓库已整理代码、README、答辩 PPT、开发流程文档和生成脚本。

后续优化方向

- 硬件优化：PCB、传感器固定、外壳结构、供电去耦与接口防松。
- 算法优化：异常点滤波、帧间平滑、热点追踪、发射率与环境温度修正。
- 功能拓展：温度极值触发 LED 或报警器，加入阈值报警和实验记录。
- 网络拓展：通过上位服务器、云端转发或内网穿透，使数据不只局限于局域网查看。
- 识别拓展：结合红外目标识别算法，进一步发展为自动识别或设备异常检测装置。
- 校准系统：加入已知温度参考源或两点标定流程，对偏移和增益进行软件补偿；该方案可行，但需要稳定参考温度和传感器固定结构配合。